

Answerspot — Python Worker Skeleton

Celery-based. Path root: `apps/workers/`. Package: `answerspot_workers`.

```
apps/workers/
├─ pyproject.toml
├─ answerspot_workers/
│   ├─ __init__.py          # Celery app
│   ├─ config.py
│   ├─ db.py                # SQLAlchemy session (read/write, no migrations)
│   ├─ schemas.py           # Pydantic contracts
│   ├─ adapters/
│   │   ├─ base.py          # PlatformAdapter interface
│   │   ├─ openai_adapter.py
│   │   ├─ perplexity_adapter.py
│   │   ├─ gemini_adapter.py
│   │   ├─ ai_overview_adapter.py  # BLOCKED — see ADR 0004
│   │   └─ registry.py
│   └─ tasks/
│       ├─ runner.py        # scan.run → query platforms
│       ├─ parser.py        # scan.parse → extract structured data
│       ├─ recommender.py    # recs.generate
│       └─ scheduler.py     # beat: enqueue scans per plan tier
```

`apps/workers/answerspot_workers/__init__.py`

python

```
from celery import Celery
from .config import settings

app = Celery(
    "answerspot_workers",
    broker=settings.celery_broker_url,
    backend=settings.celery_result_backend,
    include=[
        "answerspot_workers.tasks.runner",
        "answerspot_workers.tasks.parser",
        "answerspot_workers.tasks.recommender",
        "answerspot_workers.tasks.scheduler",
    ],
)

# Dedicated queues per pipeline stage so they scale independently (ADR 0003).
app.conf.task_routes = {
    "answerspot_workers.tasks.runner.*": {"queue": "runner"},
    "answerspot_workers.tasks.parser.*": {"queue": "parser"},
    "answerspot_workers.tasks.recommender.*": {"queue": "recs"},
}

# Beat schedule: scheduler decides which businesses are due (plan-aware).
app.conf.beat_schedule = {
    "enqueue-due-scans": {
        "task": "answerspot_workers.tasks.scheduler.enqueue_due_scans",
        "schedule": 3600.0, # hourly tick; task filters by plan cadence
    },
}

app.conf.task_acks_late = True
app.conf.task_reject_on_worker_lost = True
```

`apps/workers/answerspot_workers/config.py`

python

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    database_url: str
    celery_broker_url: str
    celery_result_backend: str

    openai_api_key: str = ""
    perplexity_api_key: str = ""
    gemini_api_key: str = ""
    serp_api_key: str = ""

    scan_sample_count: int = 3
    scan_monthly_budget_usd: float = 500.0

    sentry_dsn: str = ""

settings = Settings()
```

apps/workers/answerspot_workers/db.py

python

```
from contextlib import contextmanager
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from .config import settings

# Workers read/write the schema Prisma owns. They NEVER run migrations.
engine = create_engine(settings.database_url, pool_pre_ping=True, future=True)
SessionLocal = sessionmaker(bind=engine, expire_on_commit=False, future=True)

@contextmanager
def session_scope():
    session = SessionLocal()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

apps/workers/answerspot_workers/schemas.py

python

```
from enum import Enum
from typing import Optional
from pydantic import BaseModel, Field

class Sentiment(Enum):
    POSITIVE = "POSITIVE"
    NEUTRAL = "NEUTRAL"
    NEGATIVE = "NEGATIVE"
    UNKNOWN = "UNKNOWN"

class Citation(BaseModel):
    url: str
    title: Optional[str] = None
    source: Optional[str] = None

class CompetitorMention(BaseModel):
    name: str
    rank_position: Optional[int] = None

# Normalized output every adapter must return.
class RawAnswer(BaseModel):
    platform_key: str
    query_text: str
    text: str
    latency_ms: int
    cost_usd: float = 0.0

# Strict schema the parser LLM must produce (validated; retried on failure).
class ParsedResult(BaseModel):
    mentioned: bool
    rank_position: Optional[int] = None
    sentiment: Sentiment = Sentiment.UNKNOWN
    confidence: float = Field(ge=0.0, le=1.0, default=0.0)
    citations: list[Citation] = []
    competitors: list[CompetitorMention] = []
```

apps/workers/answerspot_workers/adapters/base.py

python

```
from abc import ABC, abstractmethod
from ..schemas import RawAnswer

class PlatformAdapter(ABC):
    """One adapter per AI platform. Returns a normalized RawAnswer.

    New platforms (incl. AI Overview once ADR 0004 clears) implement this
    interface and register in registry.py – no pipeline changes required.
    """

    key: str # matches PlatformKey enum, e.g. "PERPLEXITY"

    @abstractmethod
    def query(self, query_text: str, location: str | None = None) -> RawAnswer:
        ...
```

apps/workers/answerspot_workers/adapters/perplexity_adapter.py

python

```
import time
from .base import PlatformAdapter
from ..schemas import RawAnswer
from ..config import settings

class PerplexityAdapter(PlatformAdapter):
    key = "PERPLEXITY"

    def query(self, query_text: str, location: str | None = None) -> RawAnswer:
        prompt = query_text if not location else f"{query_text} in {location}"
        started = time.monotonic()

        # TODO(M2): call Perplexity API with settings.perplexity_api_key.
        # Keep this the FIRST real adapter – easiest official API (spec §M2).
        text = ""          # <- response content
        cost = 0.0         # <- computed from token usage for budget tracking

        return RawAnswer(
            platform_key=self.key,
            query_text=prompt,
            text=text,
            latency_ms=int((time.monotonic() - started) * 1000),
            cost_usd=cost,
        )
```

`apps/workers/answerspot_workers/adapters/ai_overview_adapter.py`

python

```

from .base import PlatformAdapter
from ..schemas import RawAnswer

class AIOverviewAdapter(PlatformAdapter):
    key = "AI_OVERVIEW"

    def query(self, query_text: str, location: str | None = None) -> RawAnswer:
        # BLOCKED: no official API. Do NOT implement in M0-M4.
        # Method (SERP API vs headless) pending legal review – see ADR 0004.
        raise NotImplementedError("AI Overview capture blocked pending ADR 0004")

```

apps/workers/answerspot_workers/adapters/registry.py

```

python

from .base import PlatformAdapter
from .perplexity_adapter import PerplexityAdapter
# from .openai_adapter import OpenAIAdapter
# from .gemini_adapter import GeminiAdapter
# from .ai_overview_adapter import AIOverviewAdapter    # blocked, ADR 0004

_ADAPTERS: dict[str, PlatformAdapter] = {
    a.key: a for a in [
        PerplexityAdapter(),
        # OpenAIAdapter(),
        # GeminiAdapter(),
    ]
}

def get_adapter(platform_key: str) -> PlatformAdapter:
    try:
        return _ADAPTERS[platform_key]
    except KeyError:
        raise ValueError(f"No adapter for platform '{platform_key}'")

```

python

```
from celery import shared_task
from celery.utils.log import get_task_logger
from ..adapters.registry import get_adapter
from ..config import settings
from ..db import session_scope
from .parser import parse_scan

log = get_task_logger(__name__)

@shared_task(
    name="answerspot_workers.tasks.runner.run_scan",
    bind=True, max_retries=3, default_retry_delay=30, acks_late=True,
)
def run_scan(self, scan_id: str, platform_key: str, query_text: str, location: str |
    """Query one platform N times (non-determinism smoothing), store raw,
    then hand off to the parser."""
    adapter = get_adapter(platform_key)
    try:
        answers = [
            adapter.query(query_text, location)
            for _ in range(settings.scan_sample_count)
        ]
    except Exception as exc:
        log.exception("scan.run failed scan=%s platform=%s", scan_id, platform_key)
        raise self.retry(exc=exc)

    with session_scope() as db:
        # TODO(M2): update Scan status=RUNNING/DONE; insert raw sample rows.
        _ = db # persist raw responses + cost per sample for budget tracking

    parse_scan.delay(scan_id=scan_id)
    return {"scan_id": scan_id, "samples": len(answers)}
```

python

```

from celery import shared_task
from celery.utils.log import get_task_logger
from pydantic import ValidationError
from ..schemas import ParsedResult
from ..db import session_scope
from .recommender import generate_recommendations

log = get_task_logger(__name__)

@shared_task(
    name="answerspot_workers.tasks.parser.parse_scan",
    bind=True, max_retries=2, default_retry_delay=15, acks_late=True,
)
def parse_scan(self, scan_id: str):
    """Extract structured data from the raw samples via an LLM constrained to
    the ParsedResult JSON schema. Aggregate samples → one result + confidence."""
    with session_scope() as db:
        # TODO(M3): load raw samples for scan_id.
        raw_samples: list[str] = []
        _ = db

    parsed: list[ParsedResult] = []
    for raw in raw_samples:
        try:
            # TODO(M3): LLM call returning JSON; validate against ParsedResult.
            parsed.append(ParsedResult.model_validate_json(raw))
        except ValidationError as exc:
            log.warning("parse invalid JSON scan=%s: %s", scan_id, exc)
            raise self.retry(exc=exc)

    # TODO(M3): aggregate parsed[] → mention rate, modal rank, sentiment,
    # confidence = agreement across samples. Write scan_result row.
    with session_scope() as db:
        _ = db

    generate_recommendations.delay(scan_id=scan_id)
    return {"scan_id": scan_id, "parsed": len(parsed)}

```

apps/workers/answerspot_workers/tasks/recommender.py

python

```
from celery import shared_task
from ..db import session_scope

@shared_task(
    name="answerspot_workers.tasks.recommender.generate_recommendations",
    acks_late=True,
)
def generate_recommendations(scan_id: str):
    """Compare presence vs competitors, detect citation/schema/review gaps,
    emit ranked recommendations with generated artifacts where possible."""
    with session_scope() as db:
        # TODO(M5): rules + artifact generation (schema snippet, copy, review
        # response draft). Also (re)compute VisibilityScore snapshot here or in
        # a dedicated task, storing weights in breakdown (ADR 0005).
        _ = db
    return {"scan_id": scan_id, "recommendations": 0}
```

apps/workers/answerspot_workers/tasks/scheduler.py

python

```
from celery import shared_task
from ..db import session_scope
from .runner import run_scan

# Plan cadence in hours (spec §5.4).
CADENCE_HOURS = {"STARTER": 24 * 7, "PRO": 24, "AGENCY": 24}

@shared_task(name="answerspot_workers.tasks.scheduler.enqueue_due_scans")
def enqueue_due_scans():
    """Hourly tick. Find active tracked queries whose org plan cadence is due
    and enqueue scan.run per enabled platform."""
    enqueued = 0
    with session_scope() as db:
        # TODO(M2): query due (business × active query × enabled platform),
        # respecting CADENCE_HOURS and last scan time.
        due: list[dict] = []
        _ = db

    for item in due:
        run_scan.delay(
            scan_id=item["scan_id"],
            platform_key=item["platform_key"],
            query_text=item["query_text"],
            location=item.get("location"),
        )
        enqueued += 1
    return {"enqueued": enqueued}
```

apps/workers/pyproject.toml

toml

```

[
project
]
name = "answerspot-workers"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "celery[redis]>=5.4",
    "sqlalchemy>=2.0",
    "psycopg[binary]>=3.2",
    "pydantic>=2.7",
    "pydantic-settings>=2.3",
    "httpx>=0.27",
    "sentry-sdk>=2.0",
]

[
project.optional-dependencies
]
dev = ["pytest>=8", "pytest-asyncio>=0.23", "ruff>=0.5", "mypy>=1.10"]

```

Pipeline flow: `scheduler.enqueue_due_scans` → `runner.run_scan` (queue: runner) → `parser.parse_scan` (queue: parser) → `recommender.generate_recommendations` (queue: recs).
Each stage is idempotent, retried with backoff, and tied to a milestone via `TODO(Mx)`.